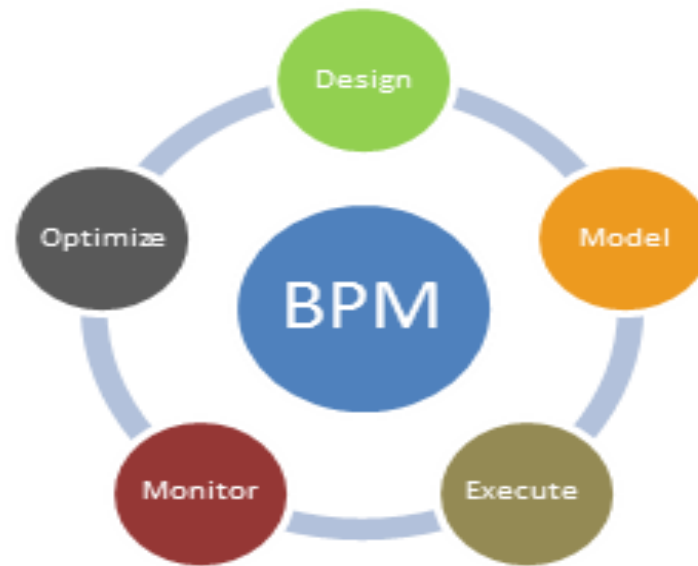


CSE346

Lect 3

Business Process Management

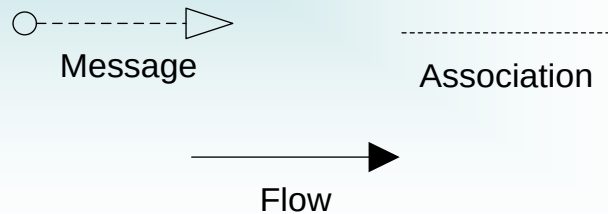


Dr. Islam El-Maddah

BPMN Continues

BPMN Main Elements - Recap

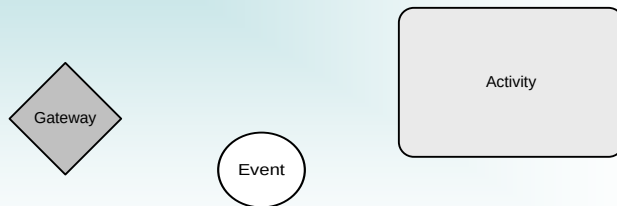
Connections



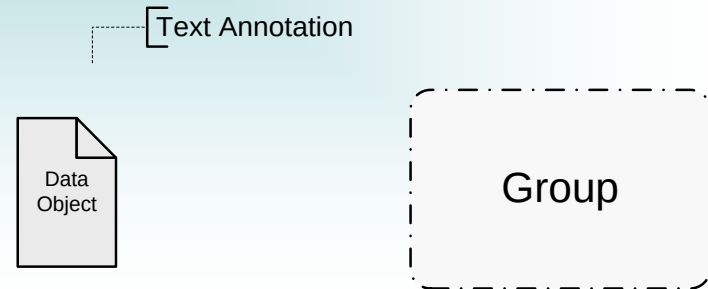
Swimlanes



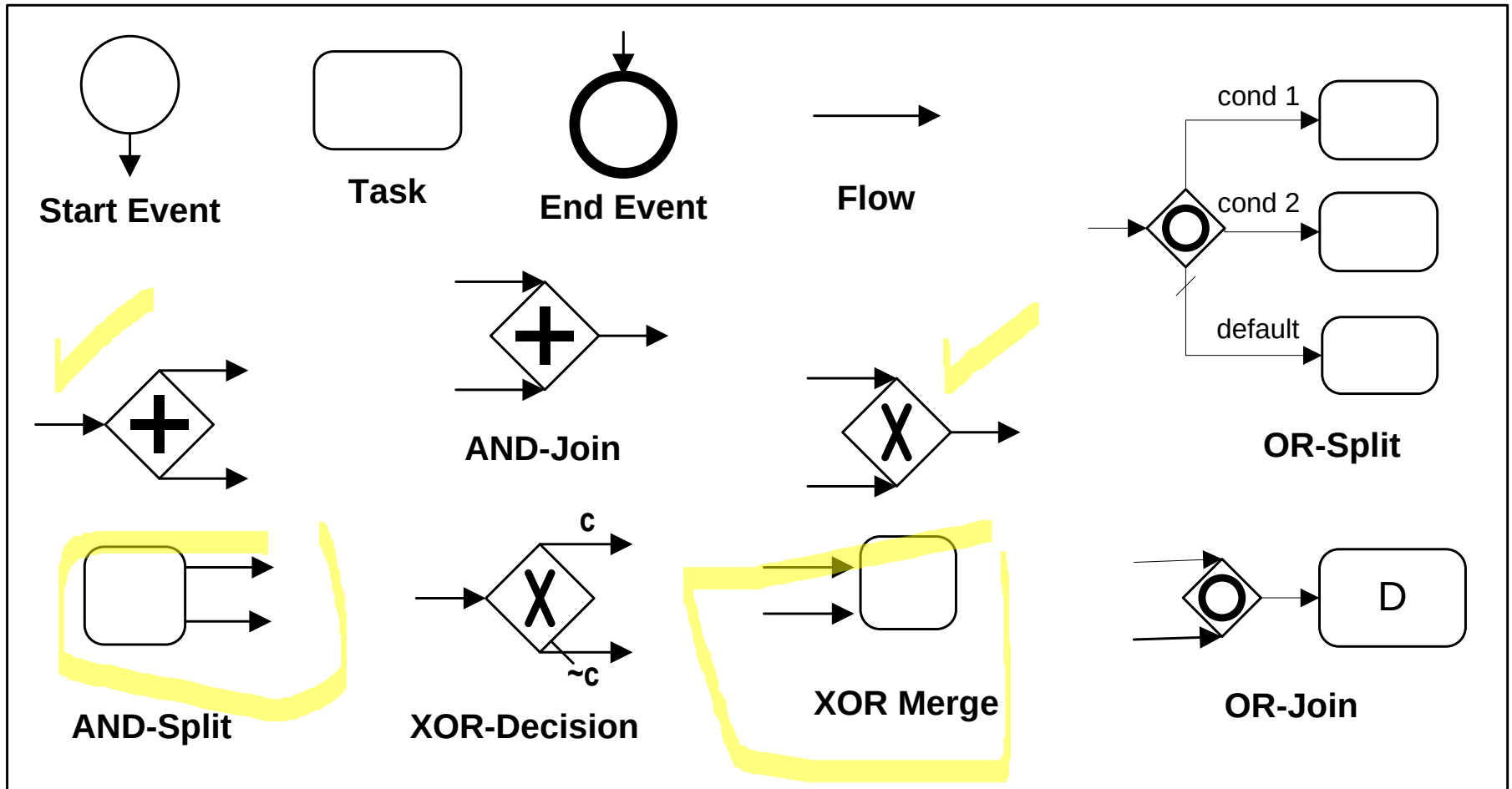
Flow Objects



Artifacts



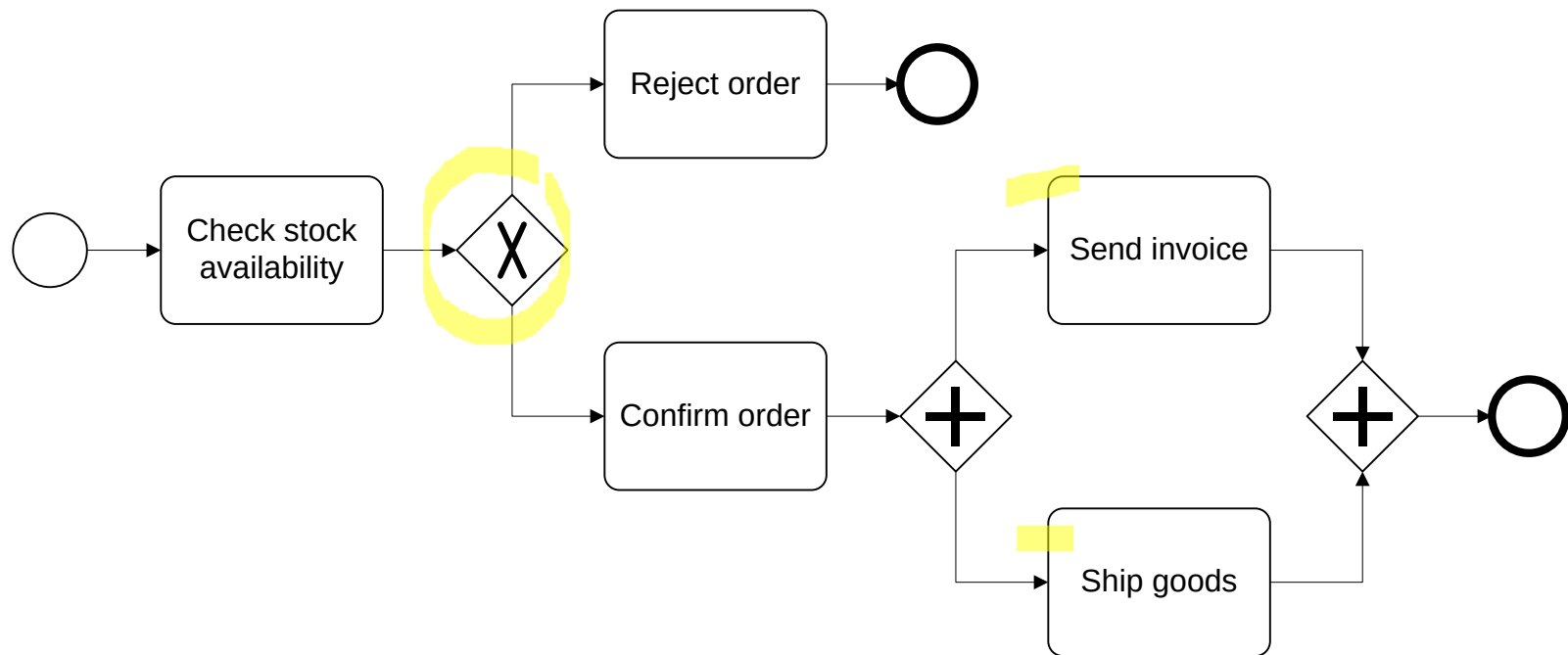
BPMN Flow Elements – Recap



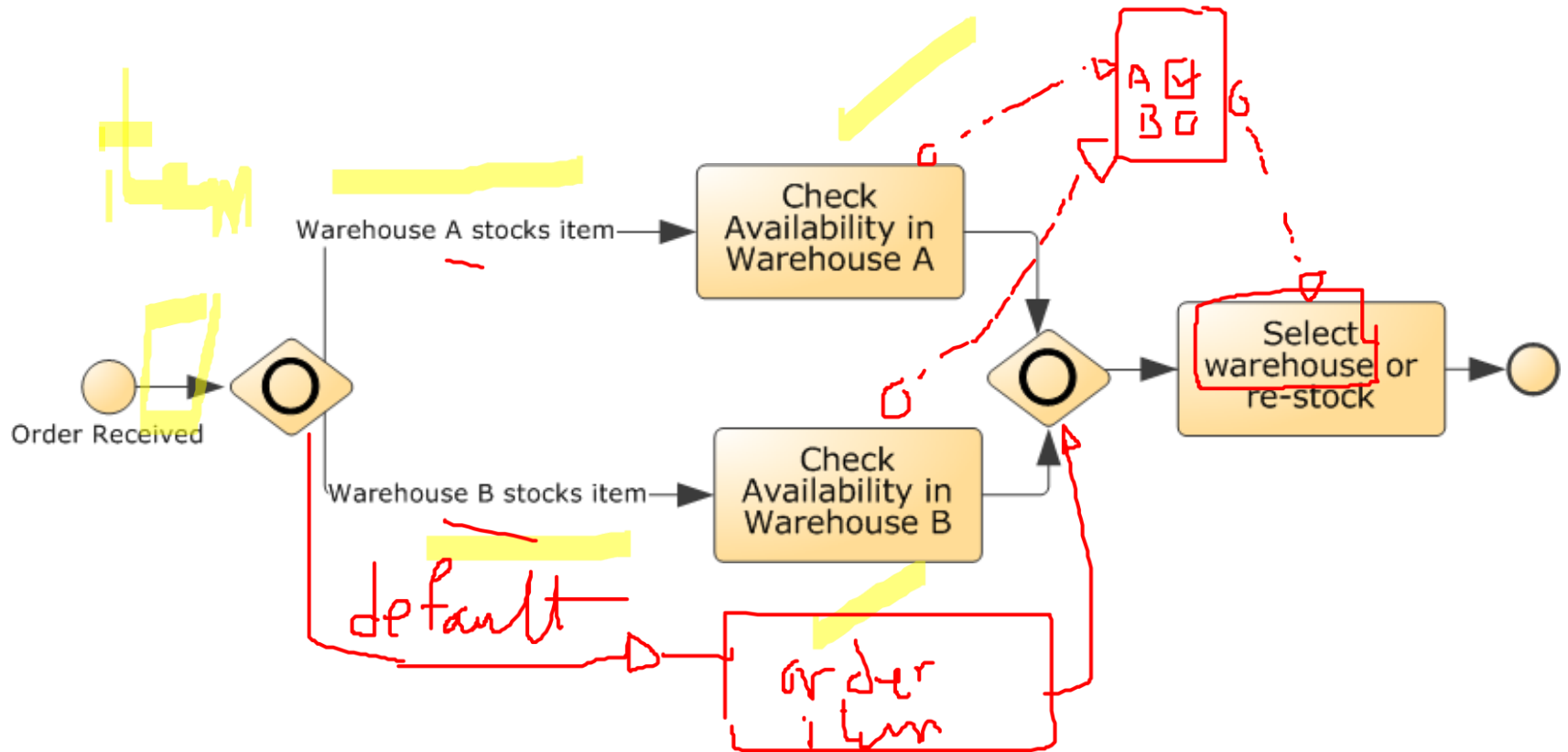
BPMN Gateways

- XOR-split (exclusive decision) → take one ~~branch~~
- XOR-merge → proceed when one branch has completed
- AND-split (fork) → take all branches
- AND-join → proceed when all incoming branches have completed
- OR-split (inclusive) → take one or several branches depending on conditions
- OR-join → proceed when all active incoming branches have completed

Example: XOR & AND gateways

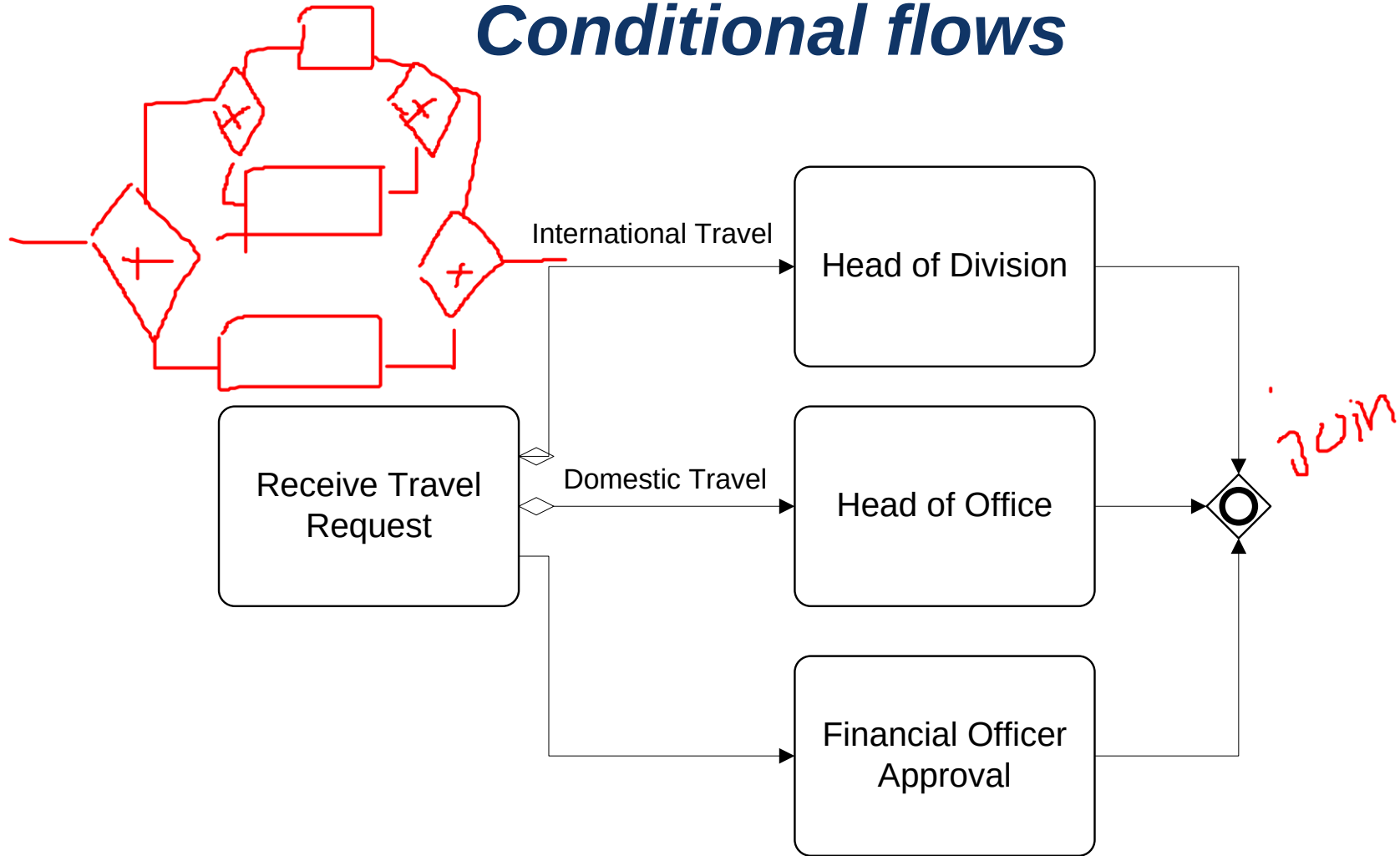


Example: OR gateways



An alternative to “split” gateways:

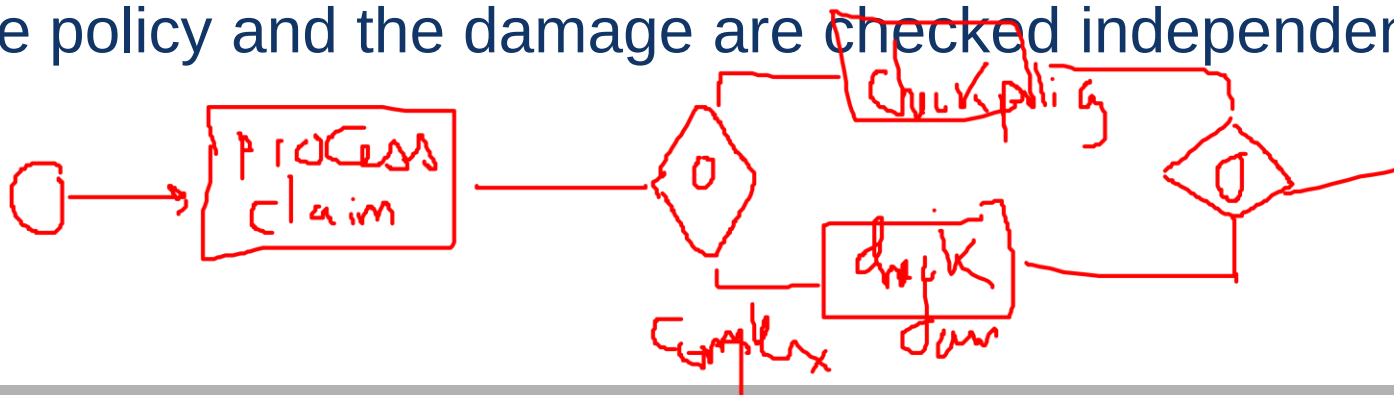
Conditional flows



BPMN Exercise 4

Map the following process fragment using OR-split, OR-join and/or conditional flows:

When a claim is received, it is registered. After registration, the claim is classified leading to two possible outcomes: simple or complex. If the claim is simple, the policy is checked. For complex claims, both the policy and the damage are checked independently.



solution

- Start Event → Claim Received
- Task → Register Claim
- Gateway X → Classify Claim (Simple or Complex)
- For Simple Claims → Check Policy
- + gateway For Complex Claims →
- Check Policy (Parallel)
- Check Damage (Parallel)
- End Event

1. Start Event → Claim Received
2. Task → Register Claim
3. OR Gateway → Classify Claim
 1. Path 1 (Complex Claim - Damage Check Only) → Check Damage
 2. Path 2 (Damage Policy for Complex and simple Claims) → Check Policy
4. End Event → Process Completed

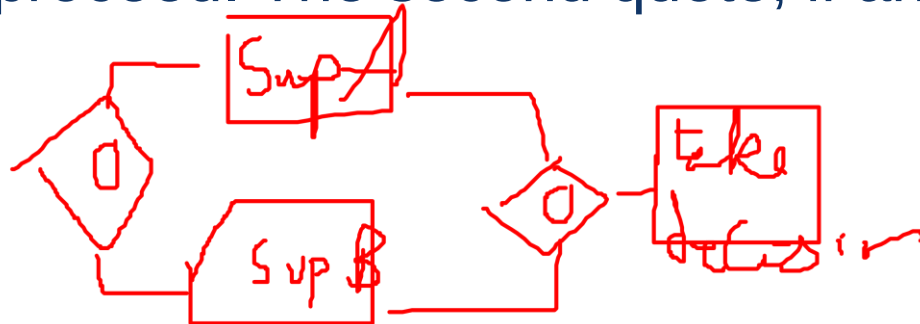
This OR Gateway allows for:

- Only Check Policy for simple claims
- Only Check Damage for certain complex claims
- Both Check Policy and Check Damage if required for complex claims

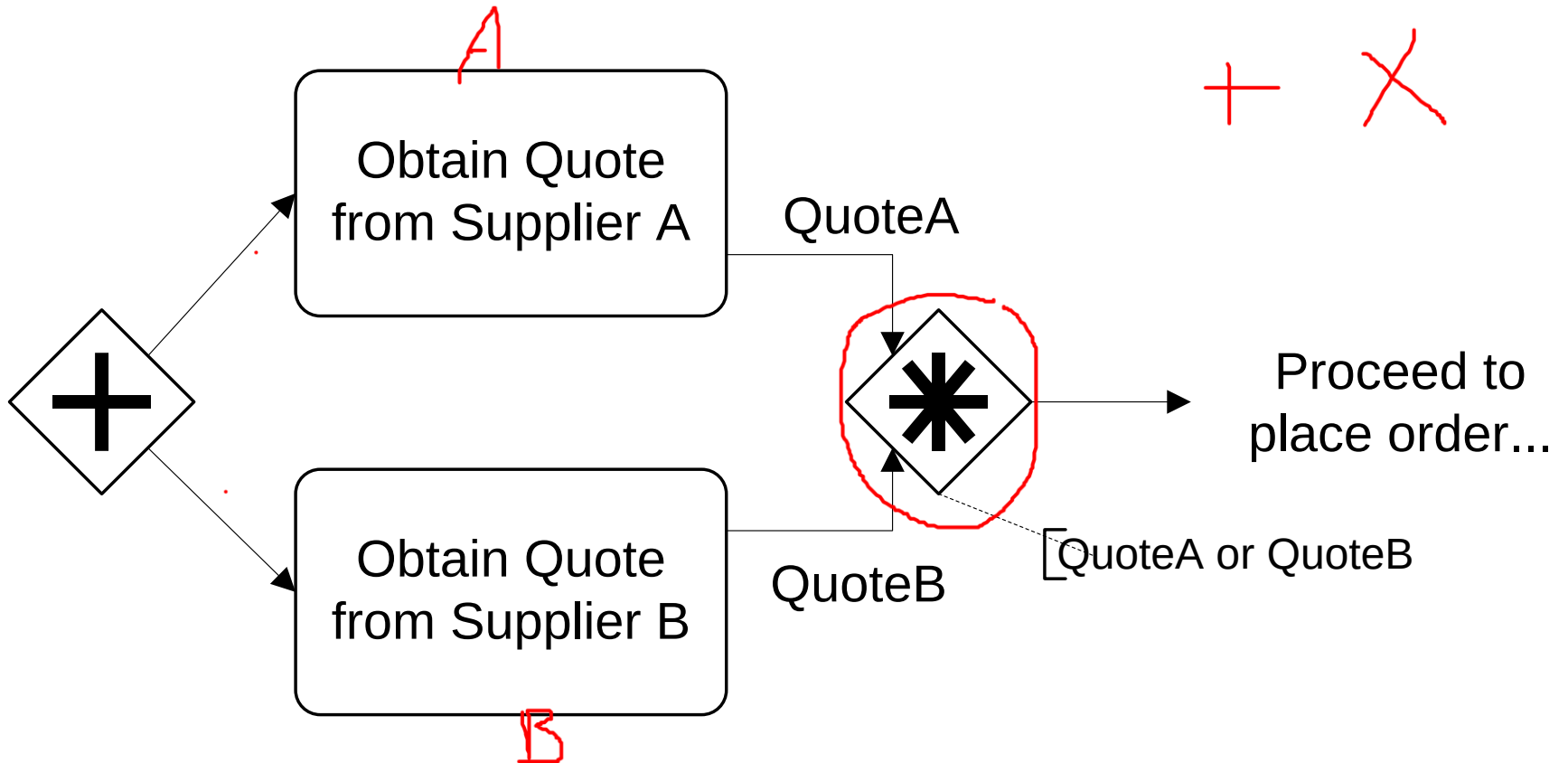
Complex gateways

- Sometimes, it is not necessary to wait for all active branches to complete:

In an order placement process, a quote is sought from two preferred suppliers in parallel. As soon as one of them provides a quote, the order placement process may proceed. The second quote, if any, is ignored.



Complex gateway example

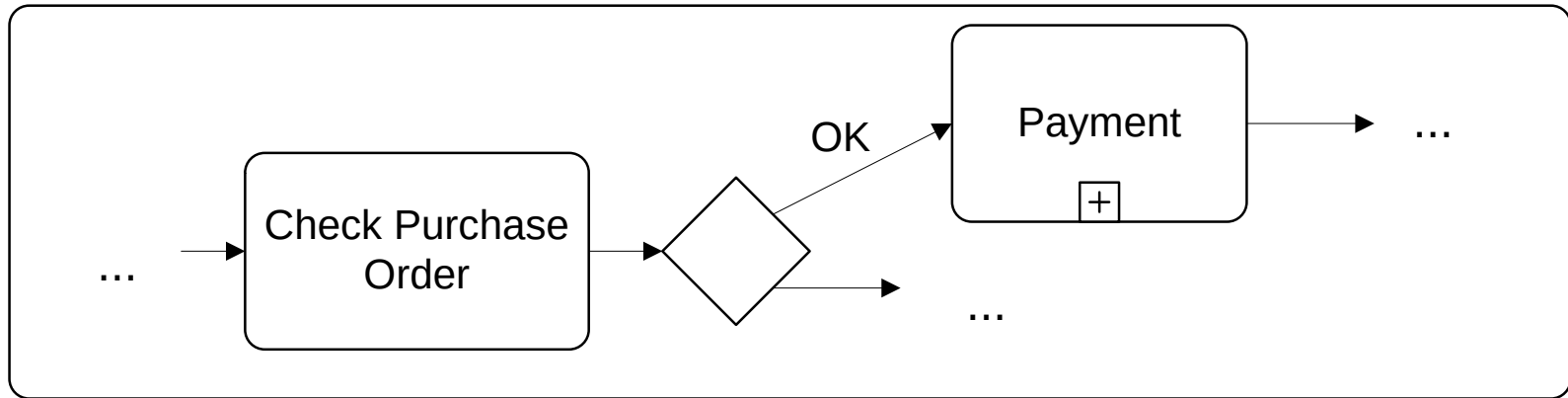


Sub-processes

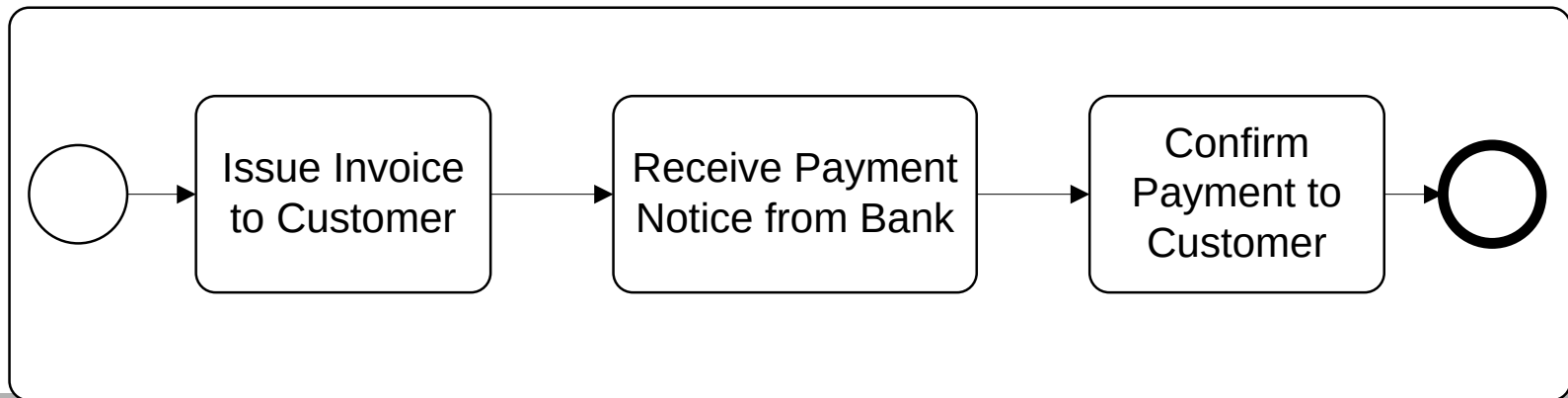
- A task in a process can be decomposed into a “sub-process”.
- Use this feature to:
 - Break down large models into smaller ones, making them easier to understand and to explain
 - Identify parts of a process model that should be:
 - repeated
 - executed multiple times in parallel
 - cancelled, or
 - compensated.

Sub-processes: example

Order Handling Process

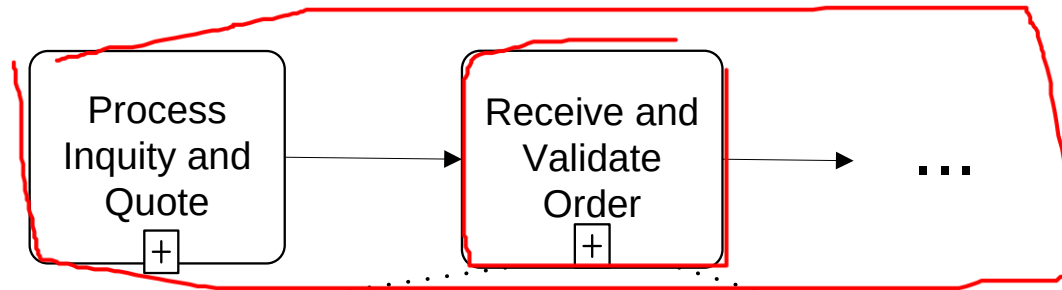


Payment Process

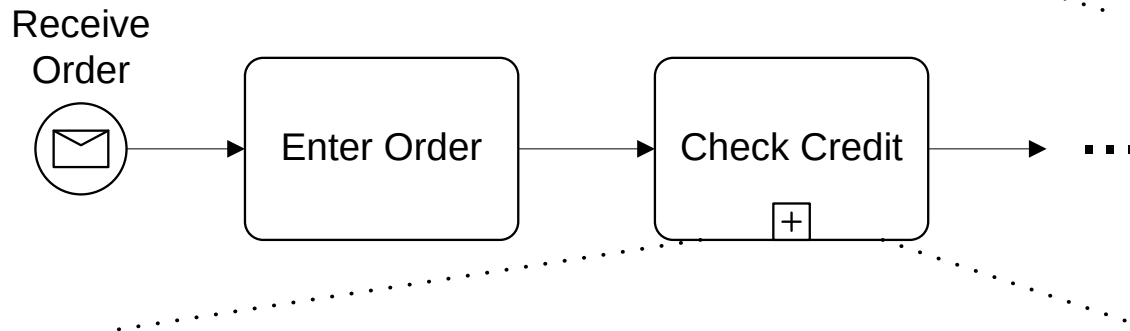


Process hierarchies

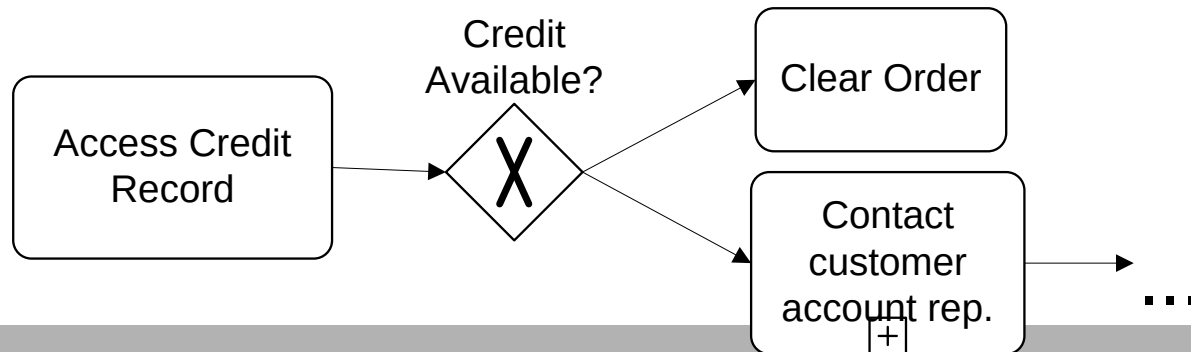
Level 3



Level 4



Level 5



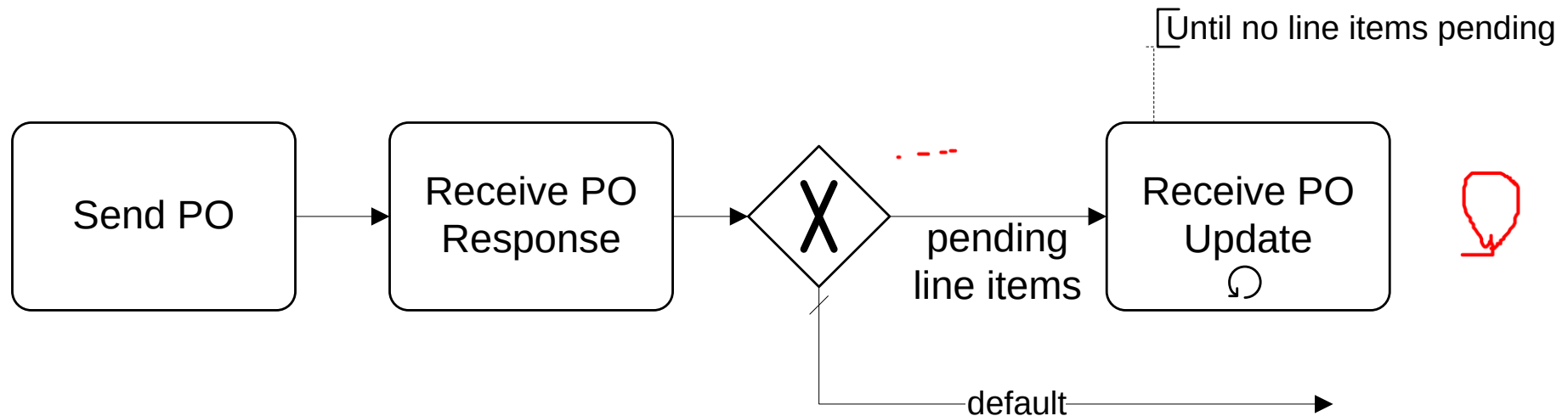
*Fragment
of the
SCOR
model*

Block-structured repetition

PO Protocol taken from www.rosettanet.com:

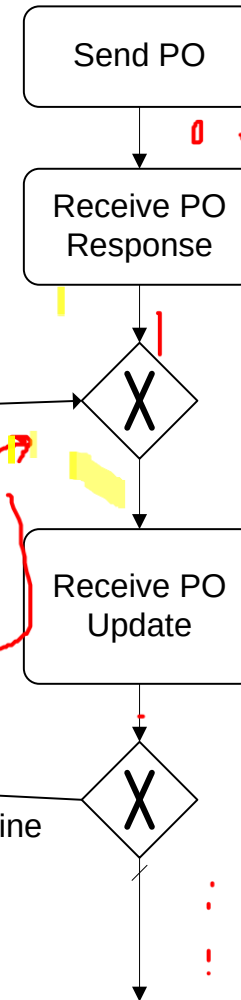
A customer sends a purchase order (PO) and expects to receive a “PO response”. In the “PO response”, each line item is marked as “accepted”, “rejected” or “pending”. If some line items are left pending, the customer expects to receive a “PO Update” later. Again, this “PO Update” marks line items as “accepted”, “rejected”, or “pending”. If some line items are left pending, the customer waits for another PO-Update, and so on until no line items are left pending.

Block-structured repetition: example



Interlude: Arbitrary cycles

- Another way of capturing “repetition” is through “cycles”: flows that go back to an “earlier” task.
- Choice between looping and cycles is often a matter of taste, but if it is possible to meaningfully cluster the activities to be repeated as a sub-process, *looping* is more suitable.



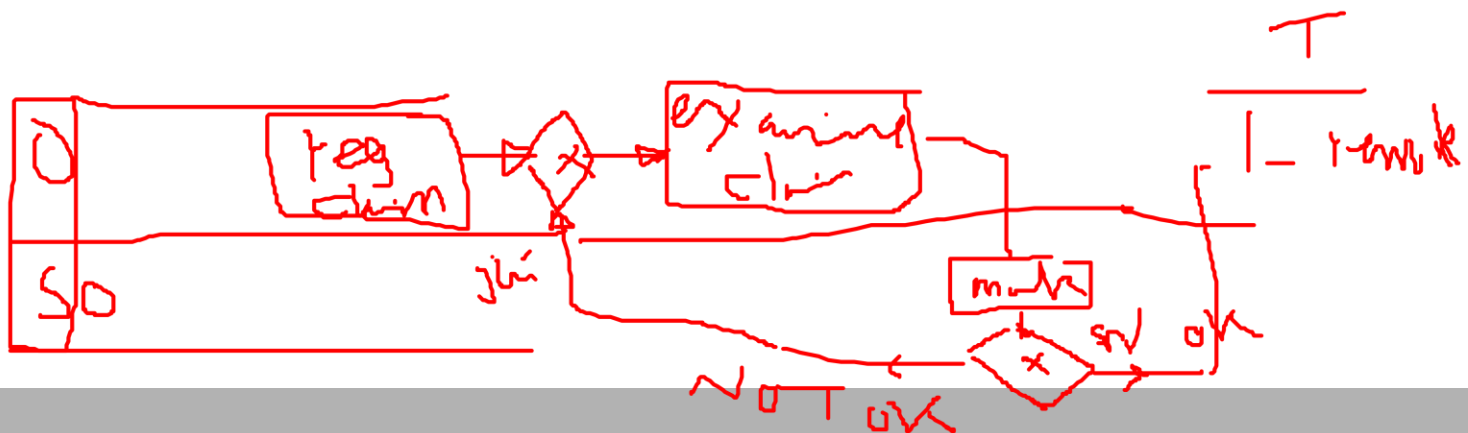
Exercise 5: Repetition/cycles

After a claim is registered, it is examined by a claims officer. The claims officer then writes a “settlement recommendation”. This recommendation is then checked by a senior claims officer who may mark the claim as “OK” or “Not OK”. If the claim is marked as “Not OK”, it is sent back to the claims officer and the examination is repeated. If the claim is OK, the claim handling process proceeds.

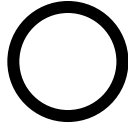
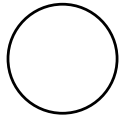
solution

- the BPMN model in a text-based format:

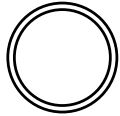
- Start Event** → Claim Registered
- Task** → Examine Claim (by Claims Officer)
- Task** → Write Settlement Recommendation
- Task** → Check Recommendation (by Senior Claims Officer)
- Exclusive Gateway (XOR)** →
 - If **OK**, proceed to **Claim Handling Process** (End Event)
 - If **Not OK**, return to **Examine Claim** (loop back to step 2)



Event types



Start/End Event – Indicates that an instance of the process is created/terminated when an event occurs without specifying the cause of this event



Intermediate event – Indicates that an event is expected to occur during the process, without indicating the cause of the event



Start Message Event – Indicates that an instance of the process is created when a **message** is received



End Message Event – Indicates that the process is terminated when a **message** is received



Intermediate Message Event – Indicates that an event is expected to occur during the process the event is triggered when a **message** is received.

Event types (cont.)



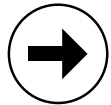
Start Timer Event – Indicates that an instance of the process is created at certain date(s)/time(s), e.g. start process at 6pm every Friday



Intermediate Timer Event – Triggered at certain date(s)/time(s), or after a time interval has elapsed since the moment the event is “enabled”



End Link Event – Indicates that the process flow continues elsewhere (e.g. in a separate diagram)



Start/Intermediate Link Event – Indicates that the process flow is being picked up from a “previous” diagram”.



Intermediate/End **Exception** Event – Indicates an error: the “end” version generates the exception event while the “intermediate” version consumes it



Intermediate/End Compensate Event – Indicates that the enclosing process must be compensated: the end version generates the compensation event while the “intermediate” version consumes it

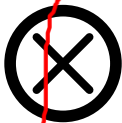
Advanced event types



Start Rule Event – Indicates that an instance of the process is created each time that a rule becomes true, e.g. start the process each time that the number of people complaining for the same reason becomes greater than 5



Intermediate Rule Event – Triggered when a rule becomes true, e.g. when an employee is sick



Intermediate/End Cancel Event – Indicates that the enclosing process must be cancelled: the end version generates the cancellation event while the “intermediate” version consumes it



End Terminate – Generates an event that causes the whole process to be terminated forcefully (i.e. nothing else should be done).

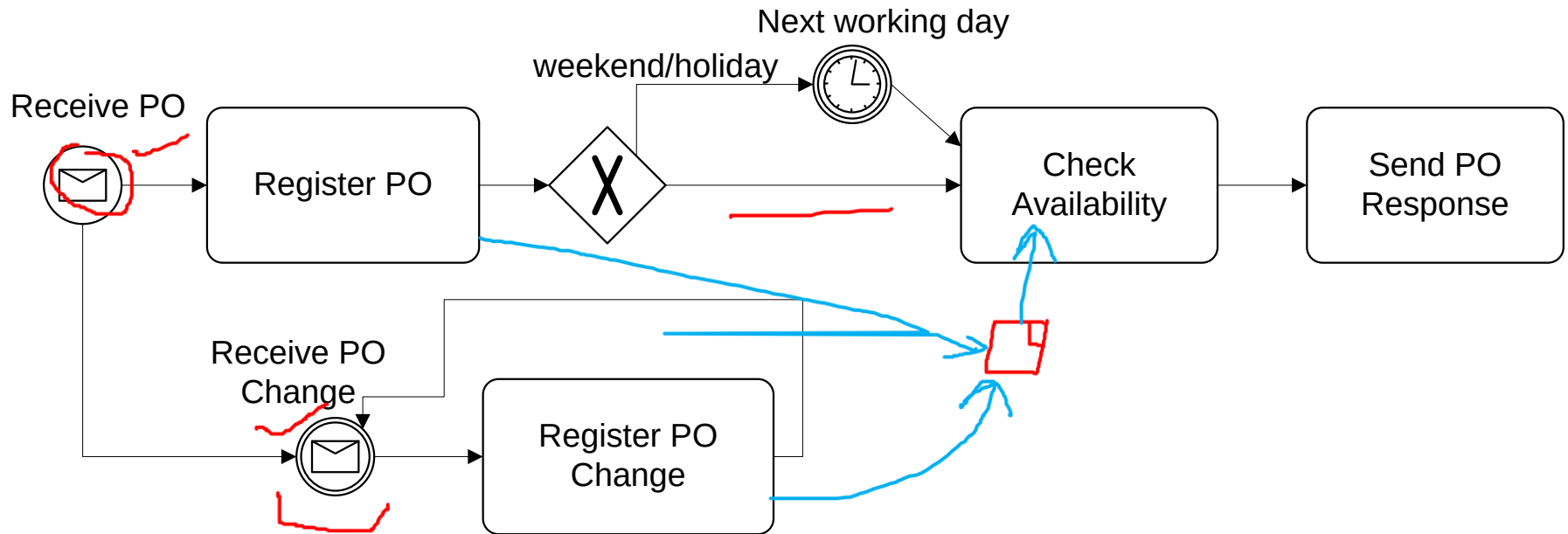
Modelling with events - Example

A PO handling process starts when a PO is received. The PO is first registered. If the current date is not a working day, the process waits until the following working day before proceeding.

Otherwise, an availability check is performed and a “PO response” is sent back to the customer.

Anytime during the process, the customer may send a “PO change request”. When such a request is received, it is just registered, without further action.

Modelling with events - Example



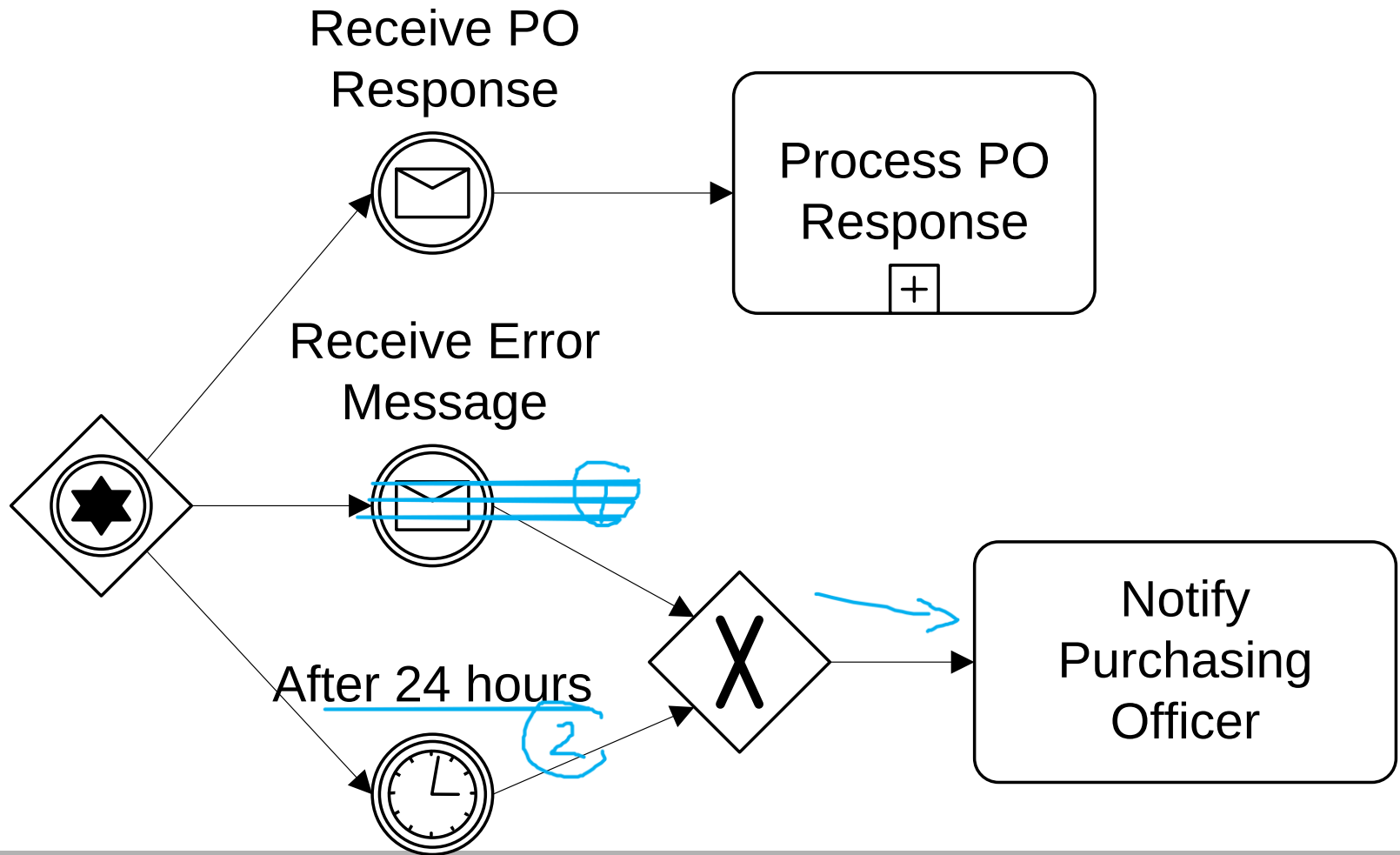
Event-based decision

- With the XOR-split gateway, a branch is chosen based on expressions that evaluate over available data
 - The choice can be made immediately after the incoming flow fires
- Sometimes, the choice must be delayed until something happens → choice is based on a race between events
- This is why BPMN distinguishes **data-driven** and **event-driven** XOR-splits

Event-based decision – Example

After a purchase order is sent, a customer can receive either a “PO Response” or an error message. It may happen that no response is received at all. If no response is received after 24 hours or if an error message is received, the purchasing officer should be notified. Otherwise, the PO Response is processed normally.

Event-based decision – Example



Exercise 5

In the context of a claim handling process, it is sometimes necessary to send a questionnaire to the claimant to gather additional information. The claimant is expected to return the questionnaire within five days. If no response is received after five days, a reminder is sent to the claimant. If after another five days there is still no response, another reminder is sent and so on until the completed questionnaire is received.

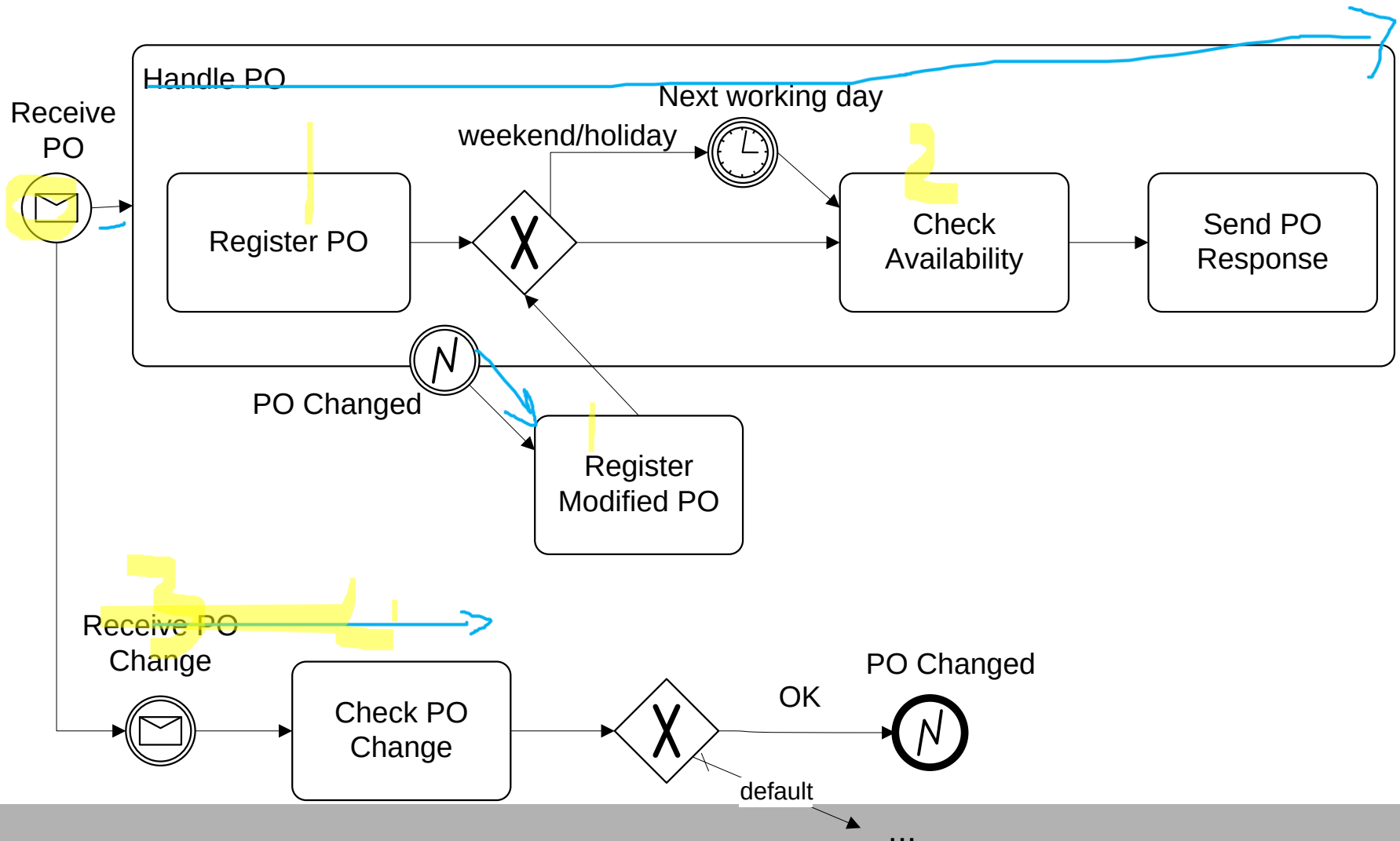
Exception handling

- Exceptions are events that deviate a process from its “normal” course
- Handling exceptions often involves stopping a sub-process and performing a special activity
- Three ways of stopping a sub-process in BPMN:
 - Simple stop: Stop a sub-process and start an exception handling routine
 - Transaction cancellation: Stop a transaction, release resources reserved during the transaction and notify any partners involved in the transaction
 - Compensation: Stop the sub-process, compensate any sub-sub-process, and run a compensation routine

Exception handling – Example

Consider the previous “PO Change Request” example with the following variation: When a PO Change Request is received, it is first checked to determined if it can be accepted. If it is accepted, any processing related to the PO must be stopped. The PO change request is then registered. Thereafter, the process proceeds as it would after a “normal” PO is registered.

Exception handling – Example



Exercise 6

Extend the claim handling process as follows:

After checking the insurance policy, a possible outcome is that the insurance is invalid. In this case, any processing is cancelled and a letter is sent to the customer. In the case of a complex claim, this implies that the damage checking is cancelled if it has not yet been completed.

Modelling conventions

Why use modelling conventions?

- To reduce variety
- Increase the comparability of models
- Support the analysis (with clear syntax, sematic and layout standards)
- Acceleration and simplification (just start and 'Go')
- ...

Different types of modelling conventions?

- Organisational level
- Project level
- ...

Modelling Guidelines

Typical categories of guidelines:

- Naming conventions for processes, tasks and events
- Guidelines for layout and usage of tasks, events, lanes and pools
- Process elements “to be avoided”

Naming conventions for processes and tasks

- Names should be 1-3 words long
- Begin with a verb followed by business object name and possibly an adjective (e.g. Issue Driver Licence, Renew Driver Licence via Offline Agencies)
- Avoid generic verbs such as Handle, Record...
- Avoid prepositions (to, from, for)
- Avoid naming business areas which are already named in a lane/pool

Verbs to avoid

- **Update, Create, Read, Delete, Record, Download, Transmit:** Too technical. Try Amend, change, generate, retrieve, remove, capture, register, forward
- **Send:** Could merely be a message flow from one business process to another.
- **Process, Handle, Manage:** Too generic, would not reflect the specific objective of the process. Try disseminate, distribute, etc.
- **Input:** Why do we have to input data? Maybe there is an opportunity for process optimisation here...

Naming conventions for events

- For start/intermediate message events: indicate what is being sent or received (e.g. Invoice Received)
- For end message events indicate what is being sent (e.g. Order Sent)
- For time events, indicate frequency or deadline, e.g. Monthly, Weakly, Invoice Due.
- Event names (except for timer events): should begin with a noun followed by a past participle
- Error detected, PO changed

Usage and layout guidelines

- A task must always have at least one incoming and one outgoing flow
- Input flows should come from the left or from the top
- Output flows should come out of the right or the bottom
- A process model should contain at least one pool
- Pools must be laid out horizontally
- A pool may appear multiple times in a diagram to improve presentation, but the name of the repeated pool must be *italicised*.
- Use *link events* to split large diagrams across multiple pages

References and acknowledgments

- Michael Hammer: “Deep Change”
- M. zur Muehlen and J. Recker: “[Process Discovery on a Rainy Day](http://tinyurl.com/cbc4uf)”. <http://tinyurl.com/cbc4uf>
- M. Rosemann. “[Pitfalls of Process Modeling](http://tinyurl.com/c4vkyg)”. Business Process Management. Journal, Volume 12, Number 2, 2006 , pp. 249-254. <http://tinyurl.com/c4vkyg>